

Llama_index RAG进阶 (Embedding_model与ChromaDB)

Llama_index RAG进阶 (Embedding_model与ChromaDB)

第一部分：Embedding Models 嵌入模型原理及选择

- 一、概念与核心原理
- 二、主流模型分类与选型指南
 1. 通用全能型
 2. 垂直领域特化型
 3. 轻量化部署型
- 三、代码案例操作

第二部分：Chroma向量数据库

- 一、Chroma 核心概念与优势
- 二、安装与基础配置
- 三、核心操作全流程

第一部分：Embedding Models 嵌入模型原理及选择

一、概念与核心原理

1. 嵌入模型的本质

嵌入模型 (Embedding Model) 是一种将离散数据 (如文本、图像) 映射到连续向量空间的技术。通过高维向量表示 (如 768 维或 3072 维)，模型可捕捉数据的语义信息，使得语义相似的文本在向量空间中距离更近。例如，“忘记密码”和“账号锁定”会被编码为相近的向量，从而支持语义检索而非仅关键词匹配。

2. 核心作用

- 语义编码**：将文本、图像等转换为向量，保留上下文信息 (如 BERT 的 CLS Token 或均值池化)。
- 相似度计算**：通过余弦相似度、欧氏距离等度量向量关联性，支撑检索增强生成 (RAG)、推荐系统等应用。
- 信息降维**：压缩复杂数据为低维稠密向量，提升存储与计算效率。

3. 关键技术原理

- 上下文依赖**：现代模型 (如 BGE-M3) 动态调整向量，捕捉多义词在不同语境中的含义。
- 训练方法**：对比学习 (如 Word2Vec 的 Skip-gram/CBOW)、预训练+微调 (如 BERT)。

二、主流模型分类与选型指南

Embedding模型将文本转换为数值向量，捕捉语义信息，使计算机能够理解和比较内容的“意义”。

选择Embedding模型的考虑因素：

因素	说明
任务性质	匹配任务需求(问答、搜索、聚类等)
领域特性	通用vs专业领域(医学、法律等)
多语言支持	需处理多语言内容时考虑
维度	权衡信息丰富度与计算成本
许可条款	开源vs专有服务
最大tokens	适合的上下文窗口大小

最佳实践：为特定应用测试多个embedding模型，评估在实际数据上的性能而非仅依赖通用基准。

1. 通用全能型

- **BGE-M3**：北京智源研究院开发，支持多语言、混合检索（稠密+稀疏向量），处理 8K 上下文，适合企业级知识库。
- **NV-Embed-v2**：基于 Mistral-7B，检索精度高（MTEB 得分 62.65），但需较高计算资源。

2.垂直领域特化型

- **中文场景**：BGE-large-zh-v1.5（合同/政策文件）、M3E-base（社交媒体分析）。
- **多模态场景**：BGE-VL（图文跨模态检索），联合编码 OCR 文本与图像特征。

3.轻量化部署型

- **nomic-embed-text**：768 维向量，推理速度比 OpenAI 快 3 倍，适合边缘设备。
- **gte-qwen2-1.5b-instruct**：1.5B 参数，16GB 显存即可运行，适合初创团队原型验。

选型决策树：

1. 中文为主 → BGE 系列 > M3E;
2. 多语言需求 → BGE-M3 > multilingual-e5;
3. 预算有限 → 开源模型（如 Nomic Embed）

三、代码案例操作

使用 HuggingFace 加载 BGE 模型进行文本嵌入

```
# 安装依赖: pip install llama-index transformers torch numpy
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
import numpy as np

# 加载 BGE 中文嵌入模型
model_name = "/home/cw/llms/embedding_model/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
embed_model = HuggingFaceEmbedding(
    model_name=model_name,
    device="cuda", # 使用 GPU, 如果没有 GPU 改为 "cpu"
    normalize=True, # 归一化向量, 方便计算余弦相似度
)

# 嵌入文档
documents = ["忘记密码如何处理?", "用户账号被锁定"]
```

```
doc_embeddings = [embed_model.get_text_embedding(doc) for doc in documents]

# 嵌入查询并计算相似度
query = "密码重置流程"
query_embedding = embed_model.get_text_embedding(query)

# 计算余弦相似度（因为 normalize=True，点积就是余弦相似度）
similarity = np.dot(query_embedding, doc_embeddings[0])
print(f"相似度: {similarity:.4f}") # 输出示例: 0.8512
```

第二部分：Chroma向量数据库

一、Chroma 核心概念与优势

1. 什么是 Chroma?

Chroma 是一款开源的向量数据库，专为高效存储和检索高维向量数据设计。其核心能力在于**语义相似性搜索**，支持文本、图像等嵌入向量的快速匹配，广泛应用于大模型上下文增强（RAG）、推荐系统、多模态检索等场景。与传统数据库不同，Chroma 基于向量距离（如余弦相似度、欧氏距离）衡量数据关联性，而非关键词匹配。

2. 核心优势

- **轻量易用**：以 Python/JS 包形式嵌入代码，无需独立部署，适合快速原型开发。
- **灵活集成**：支持自定义嵌入模型（如 OpenAI、HuggingFace），兼容 LangChain 等框架。
- **高性能检索**：采用 HNSW 算法优化索引，支持百万级向量毫秒级响应。
- **多模式存储**：内存模式用于开发调试，持久化模式支持生产环境数据落地。

二、安装与基础配置

1. 安装

通过 Python 包管理器安装 ChromaDB：

```
pip install chromadb # 完整功能
```

2. 初始化客户端

- **内存模式**（开发环境）：

```
import chromadb
client = chromadb.Client()
```

- **持久化模式**（生产环境）：

```
client = chromadb.PersistentClient(path="/path/to/save") # 数据保存至本地目录
```

三、核心操作全流程

1. 创建集合（Collection）

集合是 Chroma 中管理数据的基本单元，类似传统数据库的表：

```
collection = client.create_collection(  
    name="my_collection",  
    metadata={"hnsw:space": "cosine"}, # 指定余弦相似度计算  
    embedding_function=HuggingFaceEmbeddings(model_name="BAAI/bge-small") # 自定义嵌入模型  
)
```

2. 添加数据

支持自动生成或手动指定嵌入向量：

```
# 方式1: 自动生成向量（使用集合的嵌入模型）  
collection.add(  
    documents=["文档内容1", "文档内容2"],  
    metadatas=[{"来源": "新闻"}, {"来源": "论文"}],  
    ids=["id1", "id2"]  
)  
  
# 方式2: 手动传入预计算向量  
collection.add(  
    embeddings=[[0.1, 0.2, ...], [0.3, 0.4, ...]],  
    documents=["文本1", "文本2"],  
    ids=["id3", "id4"]  
)
```

3. 查询数据

- **文本查询**（自动向量化）：

```
results = collection.query(  
    query_texts=["查询文本"],  
    n_results=5,  
    where={"来源": "新闻"}, # 按元数据过滤  
    where_document={"$contains": "关键词"} # 按文档内容过滤  
)
```

- **向量查询**（自定义输入）：

```
results = collection.query(  
    query_embeddings=[[0.5, 0.6, ...]],  
    n_results=3  
)
```

4. 数据管理

- **更新**: `collection.update(ids=["id1"], documents=["新内容"])`
- **删除**: `collection.delete(ids=["id2"])`
- **统计**: `collection.count()` 获取条目数